[Huyen Nguyen] Linux User-Kernel Communication

Why is User-Kernel Communication Needed?

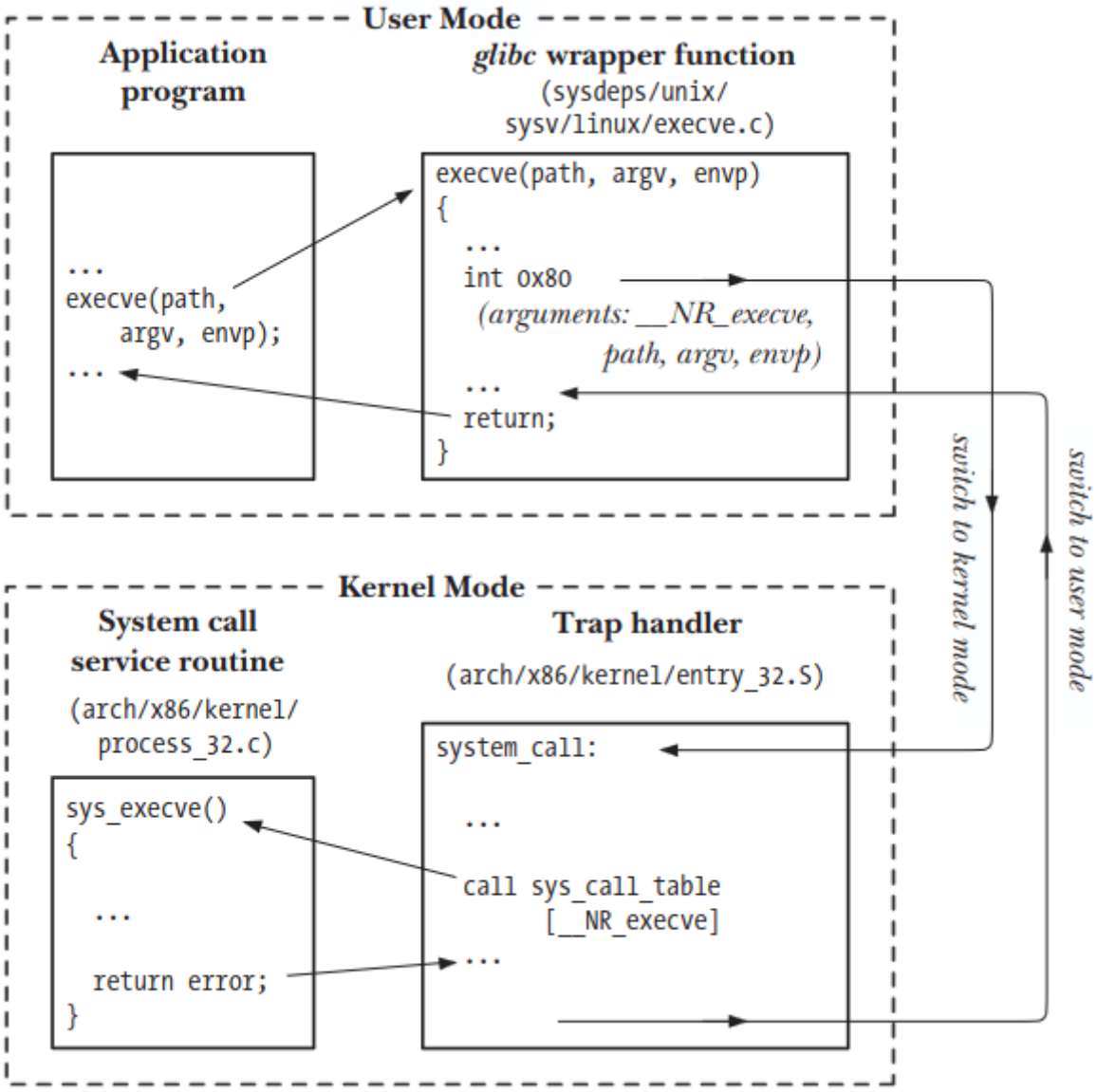
Because these two spaces have independent memory address spaces and different privilege levels. User-space processes cannot directly call kernel functions -> They must use a Standard Communication Interface.

1. System Call

The primary communication gateway of the OS, allowing user-space processes to request the kernel to perform privileged tasks on their behalf (e.g., process creation, file read/write, physical memory allocation).

- Privilege switching: Switches the CPU from user mode (Ring 3) to kernel mode (Ring 0).
- Each syscall in Linux is assigned a fixed integer number (e.g., sys_execve is 11).
- Arguments are passed from user space to the kernel, and results are returned back.

Execution Flow of a System Call



Application invocation: The C program invokes a library wrapper function (provided by glibc).

Wrapper preparation & trap activation:

- Loads the system call number into a designated CPU register (%eax on x86).
- Loads arguments into other registers (%ebx, %ecx, ...).
- Executes a machine instruction (e.g., software interrupt int 0x80 or syscall instruction) $\rightarrow$ CPU switches from user mode to kernel mode.

Kernel trap handler execution:

- Saves user-space CPU registers onto the kernel stack.
- Uses the syscall number from %eax to index into the system call table (sys_call_table) to identify the corresponding service routine.

Kernel service routine execution:

- Validates input arguments and performs the requested operation.

- Stores the return value.

Return to user mode:

- Restores saved user-space registers from the kernel stack.
- Returns execution back to the glibc wrapper function $\rightarrow$ CPU transitions back to user mode.

glibc returns result to application:

- On success: Returns the result value.
- On failure: Returns -1 and sets the global error variable errno.

Lab 1

```
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>
#include <sys/types.h>
#include <errno.h>
#include <string.h>

int main()
{
    pid_t pid_wrapper = getpid();
    pid_t pid_direct = syscall(SYS_getpid);

    printf("[1] PID:\n");
    printf("    - Via glibc wrapper: getpid()            = %d\n", pid_wrapper);
    printf("    - Direct syscall    : syscall(SYS_getpid) = %ld\n\n", (long)pid_direct);

    pid_t tid_direct = syscall(SYS_gettid);

    printf("[2] TID:\n");
    printf("    - Direct syscall    : syscall(SYS_gettid) = %ld\n\n", (long)tid_direct);

    const char *msg_wrapper = "[3] glibc wrapper: write()\n";
    const char *msg_direct  = "[3] Direct syscall: syscall(SYS_write)\n";

    write(STDOUT_FILENO, msg_wrapper, strlen(msg_wrapper));
    syscall(SYS_write, STDOUT_FILENO, msg_direct, strlen(msg_direct));
```

```
    printf("\n");

    printf("[4] Checking the error-reporting mechanism with a nonexistent syscall
(99999):\n");

    long ret = syscall(99999);

    if (ret == -1)
    {
        printf("    - Return value: %ld (Failure)\n", ret);
        printf("    - Error code  : %d (%s)\n", errno, strerror(errno));
    }

    return 0;
}
```

Output

```
[1] PID:
    - Via glibc wrapper: getpid()            = 17414
    - Direct syscall    : syscall(SYS_getpid) = 17414

[2] TID:
    - Direct syscall    : syscall(SYS_gettid) = 17414

[3] glibc wrapper: write()
[3] Direct syscall: syscall(SYS_write)

[4] Checking the error-reporting mechanism with a nonexistent syscall (99999):
    - Return value: -1 (Failure)
    - Error code  : 38 (Function not implemented)
```

Why use syscall(2) directly?

- The kernel introduces a new syscall not yet supported by the installed glibc wrapper.
- Invoking custom syscalls created during kernel customization.
- Constrained embedded systems: Avoiding large C libraries to reduce executable binary size.

2. IOCTL interface

A specialized system call: ioctl(int fd, unsigned long request, ...), allowing user applications to send custom control commands directly to a device driver.

Synchronous operation: Blocks and waits until the kernel driver completes the execution of the command.

Device-specific control: Typically used to configure and control specialized hardware features that standard system calls (like read/write) cannot accommodate.

When to use: When interacting directly with hardware devices and configuring drivers (e.g., toggling GPIO pins, setting UART baud rates, querying device status).

```
#include <sys/ioctl.h>


int ioctl(int fd, int request, ... /* argp */);
```

- int fd (file descriptor)
- int request (control command / request code):
- argp (optional argument)

Lab 2

Hardware devices or drivers are not accessed directly by hardware addresses, but are represented as files in /dev/ (called Device Nodes / Device Files). E.g.:

/dev/tty: Terminal console device

/dev/sda: Hard disk

/dev/my_device: Custom driver

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <errno.h>
#include <string.h>

int main()
{
```

```
    const char *dev_name = "/dev/tty";
    printf("[1] Opening device node: %s...\n", dev_name);

    int fd = open(dev_name, O_RDWR);

    if (fd < 0)
    {
        fprintf(stderr, "Error: Failed to open %s (%s)\n", dev_name, strerror(errno));
        return 1;
    }
    printf(" File Descriptor (fd) = %d\n\n", fd);

    struct winsize ws;
    memset(&ws, 0, sizeof(ws));

    printf("[2] Sending ioctl(fd, TIOCGWINSZ, &ws)...\n");
    int ret = ioctl(fd, TIOCGWINSZ, &ws);

    if (ret < 0)
    {
        fprintf(stderr, "Error calling ioctl: %s\n", strerror(errno));
        close(fd);
        return 1;
    }

    printf("Rows    : %d\n", ws.ws_row);
    printf("Columns : %d\n", ws.ws_col);
    printf("X pixel : %d px\n", ws.ws_xpixel);
    printf("Y pixel : %d px\n\n", ws.ws_ypixel);
    printf("[3] Closing device node...\n");
    close(fd);
    return 0;
}
```

Output

```
[1] Opening device node: /dev/tty...
 File Descriptor (fd) = 3

[2] Sending ioctl(fd, TIOCGWINSZ, &ws)...
```

```
Rows : 15
Columns  : 119
X pixel : 0 px
Y pixel : 0 px

[3] Closing device node...
```

What happens in the kernel when open() is invoked?

The Virtual File System (VFS) resolves the path in the /dev/ directory and looks up the "tty" inode.

The kernel identifies "tty" as a character device associated with the device numbers (Major:Minor):

- Major number = 5: Identifies the core driver subsystem (Major 5 maps to the TTY driver, whereas Major 8 would map to SCSI disk drivers).
- Minor number = 0: Identifies the specific sub-device/instance (a single TTY driver manages

The kernel allocates and initializes a 'struct file', binds it to the driver's file operations, and assigns it to an unused file descriptor (fd) in the process's descriptor table.

Returns the allocated file descriptor (fd) back to user space.

Why call open()?

ioctl() must bind to a specific device to issue control commands.

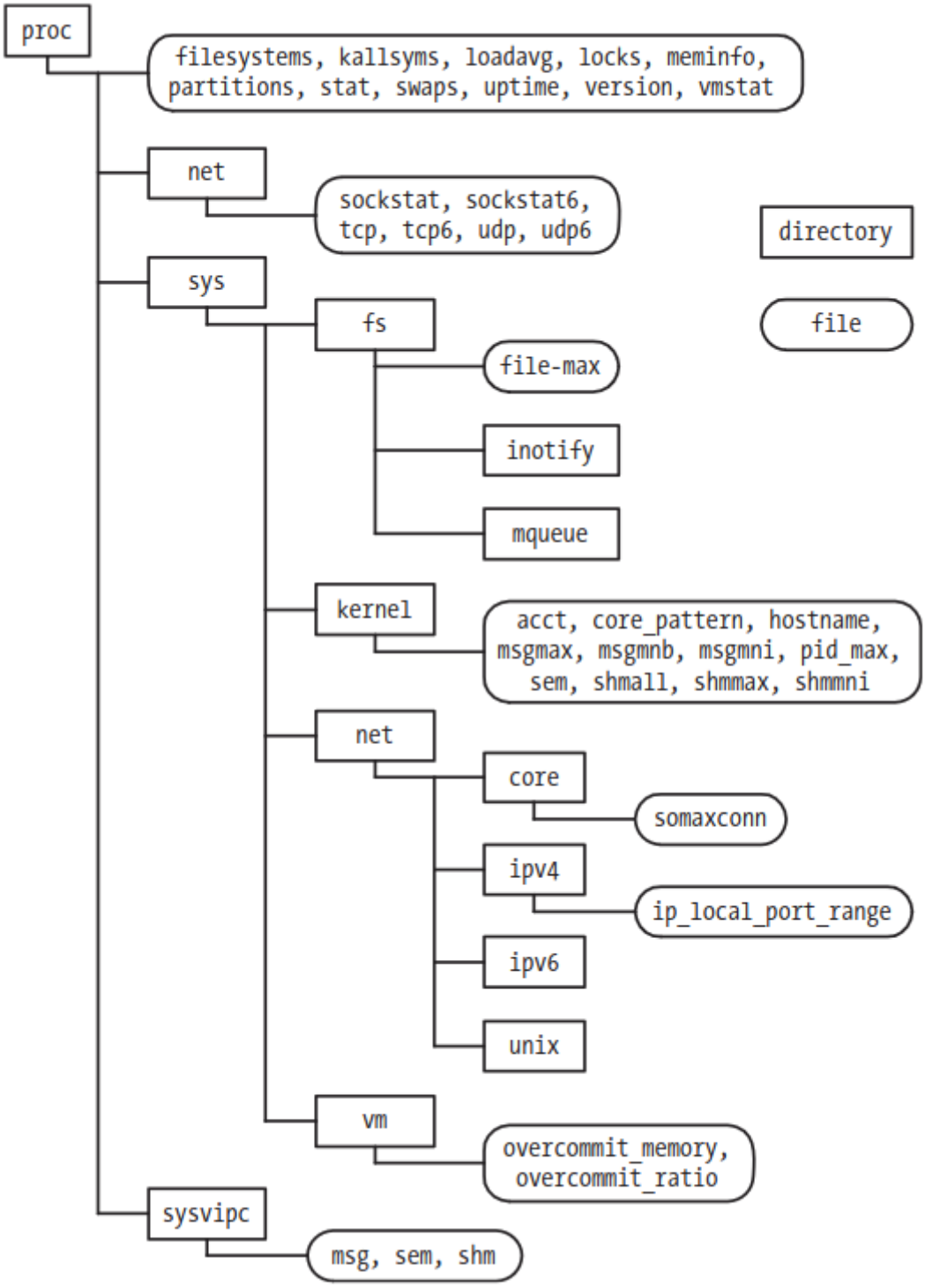
open() retrieves a file descriptor (fd) to specify which hardware driver to interact with.

3. proc file system interface

A Virtual File System (residing in RAM) used to view internal kernel states and system statistics.

For each running program, the kernel automatically creates a dedicated sub-directory /proc/<PID> to expose process-specific information.

Directory tree and files under /proc



Branch 1: Files residing directly at the root (system information files — read-only).

Branch 2: The /proc/net/ directory (network and socket status).

Branch 3: The /proc/sys/ directory (kernel runtime configuration / sysctl tuning).

fs/ directory (File System):

- Writable files used to tune and alter operating system file-system behaviors.

kernel/ directory:

- pid_max: Maximum Process ID value that the OS can allocate.
- hostname: System host name (can be dynamically updated by writing to this file).
- core_pattern: Template path defining how core dump files are named and stored.

net/ directory (Network Configuration):

- core/somaxconn: Maximum limit for the socket listen backlog queue.
- ipv4/ip_local_port_range: Dynamic ephemeral port range (e.g., from 32768 to 60999).

vm/ directory (Virtual Memory):

- overcommit_memory: Virtual memory allocation policy (determines whether the kernel allows overcommitting virtual memory beyond available physical RAM and swap).

Branch 4: The /proc/sysvipc/ directory (System V Inter-Process Communication — IPC).

Exposes monitoring and accounting information for System V IPC mechanisms:

- shm: Information on shared memory segments.
- sem: Information on semaphores.
- msg: Information on message queues.

Lab 3

Reading data from kernel space to userspace

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>

#define BUFFER_SIZE 512

int main()
{
    const char *proc_file = "/proc/version";
    char buffer[BUFFER_SIZE];

    printf(" Opening file %s...\n", proc_file);
    int fd = open(proc_file, O_RDONLY);
    if (fd == -1)
    {
```

```
        fprintf(stderr, "Error opening file: %s (%s)\n", proc_file, strerror(errno));
        return 1;
    }
    memset(buffer, 0, sizeof(buffer));
    ssize_t bytes_read = read(fd, buffer, sizeof(buffer) - 1);
    if (bytes_read == -1)
    {
        fprintf(stderr, "Error reading file: %s\n", strerror(errno));
        close(fd);
        return 1;
    }

    printf("\n--- KERNEL RETURN CONTENT (%zd bytes) ---\n", bytes_read);
    printf("%s", buffer);

    close(fd);
    printf("File closed successfully.\n");
    return 0;
}
```

Output

```
 Open file /proc/version...

--- KERNEL RETURN CONTENT (226 bytes) ---
Linux version 7.0.0-38-generic (buildd@lcy02-amd64-063) (x86_64-linux-gnu-gcc-13 (Ubuntu
13.3.0-6ubuntu2~24.04.1) 13.3.0, GNU ld (GNU Binutils for Ubuntu) 2.42) #38~24.04.4-Ubuntu SMP
PREEMPT_DYNAMIC Mon Sep 14 16:37:11 UTC 2
File closed successfully.
```

Writing data from kernel space to userspace

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Please provide content to write...\n");
```

```
        return 1;
    }

    const char *proc_entry = "/proc/hello_proc";

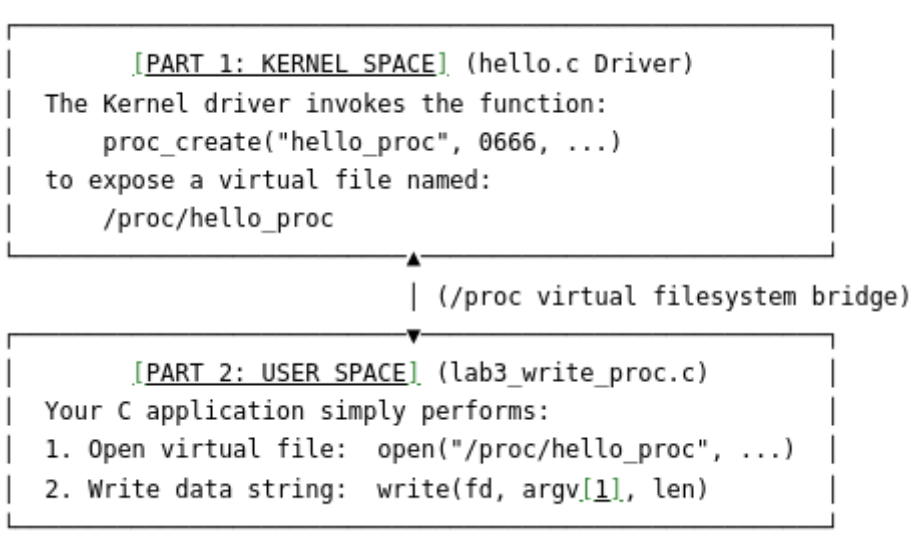
    int fd = open(proc_entry, O_WRONLY);
    if (fd < 0) {
        perror("Error opening /proc file");
        return 1;
    }

    write(fd, argv[1], strlen(argv[1]));
    printf("Successfully wrote: %s\n", argv[1]);

    close(fd);
    return 0;
}
```

Output

```
[ 7382.667698] hello: loading out-of-tree module taints kernel.
[ 7382.667704] hello: module verification failed: signature and/or required key missing - tainting kernel
[ 7382.667955] === HELLO MODULE INIT ===
[ 7427.411229] Driver da nhan duoc chuoi: Hello Linux Kernel!
[ 7465.449509] Driver da nhan duoc chuoi: WritingWriting
```



4. Netlink Socket

A socket-based IPC mechanism for bidirectional communication between the kernel and user space.

Operates in a message-oriented / datagram-based manner.

```
int fd = socket(AF_NETLINK, SOCK_RAW, netlink_family);
```

- AF_NETLINK: The Netlink socket address family / domain.
- netlink_family: The target kernel subsystem / protocol family to interact with.

Packet header structure: struct nlmsghdr

```
struct nlmsghdr {
    __u32 nlmsg_len;   /* Total length of the message = sizeof(struct nlmsghdr) + payload size
*/
    __u16 nlmsg_type;  /* Message type / command (e.g., RTM_GETLINK, NLMSG_DONE, NLMSG_ERROR)
*/
    __u16 nlmsg_flags; /* Operational behavior flags (e.g., NLM_F_REQUEST, NLM_F_DUMP) */
    __u32 nlmsg_seq;   /* Sequence number (custom tracking number for request-response
pairing) */
    __u32 nlmsg_pid;   /* Port ID of the sending process (typically set to getpid() or 0) */
};
```

NETLINK_ROUTE

- Used to configure and monitor networking: routing tables (ip route), IP addresses (ip addr), network interfaces (ip link).

Why is Netlink well-suited for network applications?

The Networking Subsystem resides in the Kernel

- Wi-Fi interfaces, Ethernet network interface cards (NICs).
- In-flight network packets.
- IP addresses, active connection states, packet routing tables.

Userspace networking utilities (ip, ss, NetworkManager)

Run in user space

- ip link set eth0 up (brings up a network interface).
- ss -tuln (inspects currently listening ports).

- NetworkManager (manages connections dynamically).

-> These utilities need to communicate with the Kernel.

-> To enable this communication, Linux establishes NETLINK as the standard communication protocol.

Why choose NETLINK over ioctl or /proc?

If using ioctl:

- ioctl was designed for controlling individual hardware devices.
- Networking is far more complex: a single interface can carry multiple IP addresses, dynamic routes, traffic control policies, etc., which exceed fixed ioctl structures.

If using /proc: Files under /proc are unidirectional text streams. They are primarily read-only and inefficient for issuing complex, dynamic runtime configurations.

NETLINK solves these challenges via a SOCKET interface:

- Bidirectional communication: Supports full two-way request/response exchanges.
- Asynchronous event notifications from the Kernel: When an Ethernet cable is unplugged, the Kernel proactively broadcasts a Netlink message to NetworkManager. As a result, the network icon on the desktop turns into a red "X" immediately, without the application having to poll the Kernel continuously.

Lab 4

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <linux/netlink.h>
#include <linux/rtnetlink.h>
#include <net/if.h>

int main()
{
    int fd = socket(AF_NETLINK, SOCK_RAW, NETLINK_ROUTE);

    struct
    {
        struct nlmsghdr  nlh;
        struct ifinfomsg ifi;
```

```
}
req =
{
    .nlh.nlmsg_len   = sizeof(req),
    .nlh.nlmsg_type  = RTM_GETLINK,
    .nlh.nlmsg_flags = NLM_F_REQUEST | NLM_F_DUMP,
};

send(fd, &req, sizeof(req), 0);

char buf[4096];
int len = recv(fd, buf, sizeof(buf), 0);

printf("%-5s %-16s %s\n", "INDEX", "INTERFACE", "STATUS");
printf("-----------------------------------\n");

for (struct nlmsghdr *nlh = (struct nlmsghdr *)buf;
     NLMSG_OK(nlh, len);
     nlh = NLMSG_NEXT(nlh, len))
{
    if (nlh->nlmsg_type == NLMSG_DONE) break;

    if (nlh->nlmsg_type == RTM_NEWLINK)
    {
        struct ifinfomsg *ifi = NLMSG_DATA(nlh);
        char *ifname = "unknown";
        int rta_len = IFLA_PAYLOAD(nlh);
        for (struct rtattr *rta = IFLA_RTA(ifi);
             RTA_OK(rta, rta_len);
             rta = RTA_NEXT(rta, rta_len))
        {
            if (rta->rta_type == IFLA_IFNAME)
            {
                ifname = (char *)RTA_DATA(rta);
            }
        }
        printf("%-5d %-16s %s\n",
               ifi->ifi_index,
               ifname,
               (ifi->ifi_flags & IFF_UP) ? "UP" : "DOWN");
```

```
        }
    }

    close(fd);
    return 0;
}
```

Output

```
INDEX INTERFACE        STATUS
-----------------------------------
1     lo               UP
2     wlo1             UP
```

Workflow

- Open a NETLINK_ROUTE socket and send an RTM_GETLINK command with NLM_F_DUMP to request all network interfaces.
- The Kernel dumps binary messages into the buf buffer.
- Iterate using standard macros (NLMSG_* for messages, RTA_* for attributes) to extract Index, Interface Name, and UP/DOWN status.

Revision #1
Created 2026-10-19 01:48:36 UTC by Huyen Nguyen
Updated 2026-10-19 10:27:16 UTC by Huyen Nguyen